CS & IT ENGINEERING



COMPUTER ORGANIZATION AND ARCHITECTURE

Instruction and Addressing Modes



Lecture No.- 05

By- Vishvadeep Gothi sir

Recap of Previous Lecture



Topic

Instruction

Topics to be Covered



- Topic: Effective Address
- Topic: Branch Instruction
- Topic: Instruction Cycle
- Topic: Fetch Cycle & Execution Cycle
- Topic: Addressing Modes

[NAT]

#Q. In a simplified computer the instructions are:

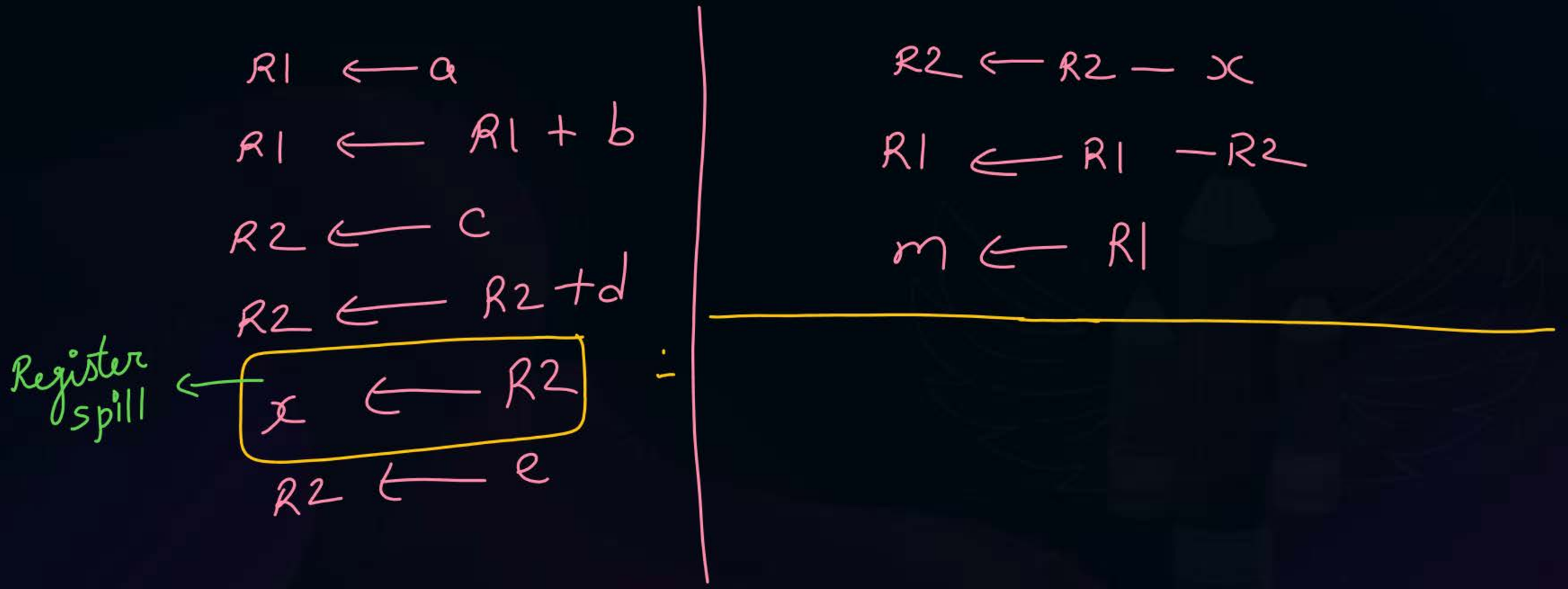
OP R_i , R_j	- Performs R_i Op R_j and stores the result in R_i
OP R_i, m	- Performs R_i Op val and stores the result in R_i val denotes the content of memory location m
MOV m , R_i	- Moves the content of memory location m to register R_i
MOV R_i , m	- Moves the content of register R_i to memory location m

The computer has only two registers and OP is either ADD or SUB. Consider the following basic block:

R1 t1 = a + b
R2 t2 = c + d
t3 = e – t2
t4 = t1 – t3

[NAT]

#Q. Assume that all operands are initially in memory. The final value of the computation should be in memory. What is the minimum number of MOV instructions in the code generated for this basic block?





Topic : Register Spill

If sufficient number of registers are not there in CPU and if to bring another operand in register, a register value is copied to memory for temporary basis.

[NAT]

Ans = 2

#Q. Consider a register-memory architecture system (2-address instructions). For this system the following intermediate code is going to be converted in machine code. Minimum how many registers are required in system so that the code can run without register spill?

Consider first operand is always the register operand and it's the destination for operation too.

t1 = X + Y

t2 = t1 – Z

t3 = t1 + t2

t4 = M + t3

Assume X, Y, Z and M are memory operands

ADD	Reg to mem
SUB	mem to Reg.
MUL	Reg to Reg.

$R1 \leftarrow X$

$R1 \leftarrow R1 + Y$

$R2 \leftarrow R1$

$R1 \leftarrow R1 - Z$

$R2 \leftarrow R2 + R1$

$R1 \leftarrow M$

$R1 \leftarrow R1 + R2$

[NAT]

Ans = 3

#Q. Consider a register-based architecture system (2-address instructions). For this system the following intermediate code is going to be converted in machine code. Minimum how many registers are required in system so that the code can run without register spill?

Consider first operand is always the register operand and it's the destination for operation too.

t1 = X + Y
t2 = t1 – Z
t3 = t1 + t2
t4 = M + t3

Assume X, Y, Z and M are memory operands

R1 ← X
R2 ← Y
R1 ← R1 + R2
R2 ← R1
R3 ← Z
R1 ← R1 − R3
R2 ← R2 + R1
R1 ← M

R1 ← R1 + R2

[NAT] GATE-2020

#Q. A processor has 64 registers and uses 16-bit instruction format. It has two types of instructions: I-type and R-type. Each I-type instruction contains an opcode, a register name, and a 4-bit immediate value. Each R-type instruction contains an opcode and two register names. If there are 8 distinct I-type opcodes, then the maximum number of distinct R-type opcodes is __14__?

[NAT] GATE-2018

#Q. A processor has 16 integer registers (R0, R1, … , R15) and 64 floating point registers (F0, F1, … , F63). It uses a 2-byte instruction format. There are four categories of instructions: Type-1, Type-2, Type-3, and Type 4. Type-1 category consists of four instructions, each with 3 integer register operands (3Rs). Type-2 category consists of eight instructions, each with 2 floating point register operands (2Fs). Type-3 category consists of fourteen instructions, each with one integer register operand and one floating point register operand (1R+1F). Type-4 category consists of N instructions, each with a floating-point register operand (1F).

The maximum value of N is 32 ?



Topic : Effective Address

- Address of operand in a computation-type instruction or
- The target address in a branch-type instruction.





Topic : Branch Instruction

Assume CPU is currently executing inst^n I2

So, PC = 204

Assume I2 is a branch instruction

(Branch taken) → Condition True

Assume ⇒ branch taken to Inst^n I7

Target inst^n ⇒ I7

Target address ⇒ 212

PC updated by Value = 212

(Branch not taken) → Condition false

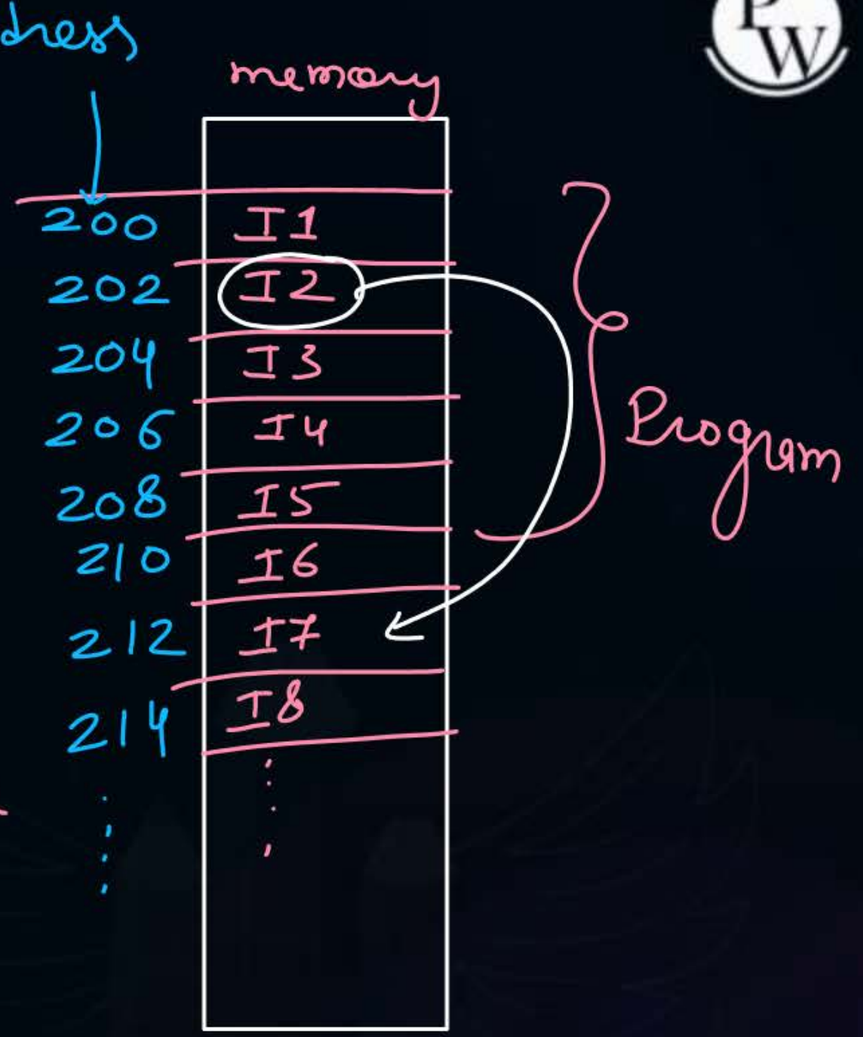
Next inst^n in the sequence should execute next

⇓

I3

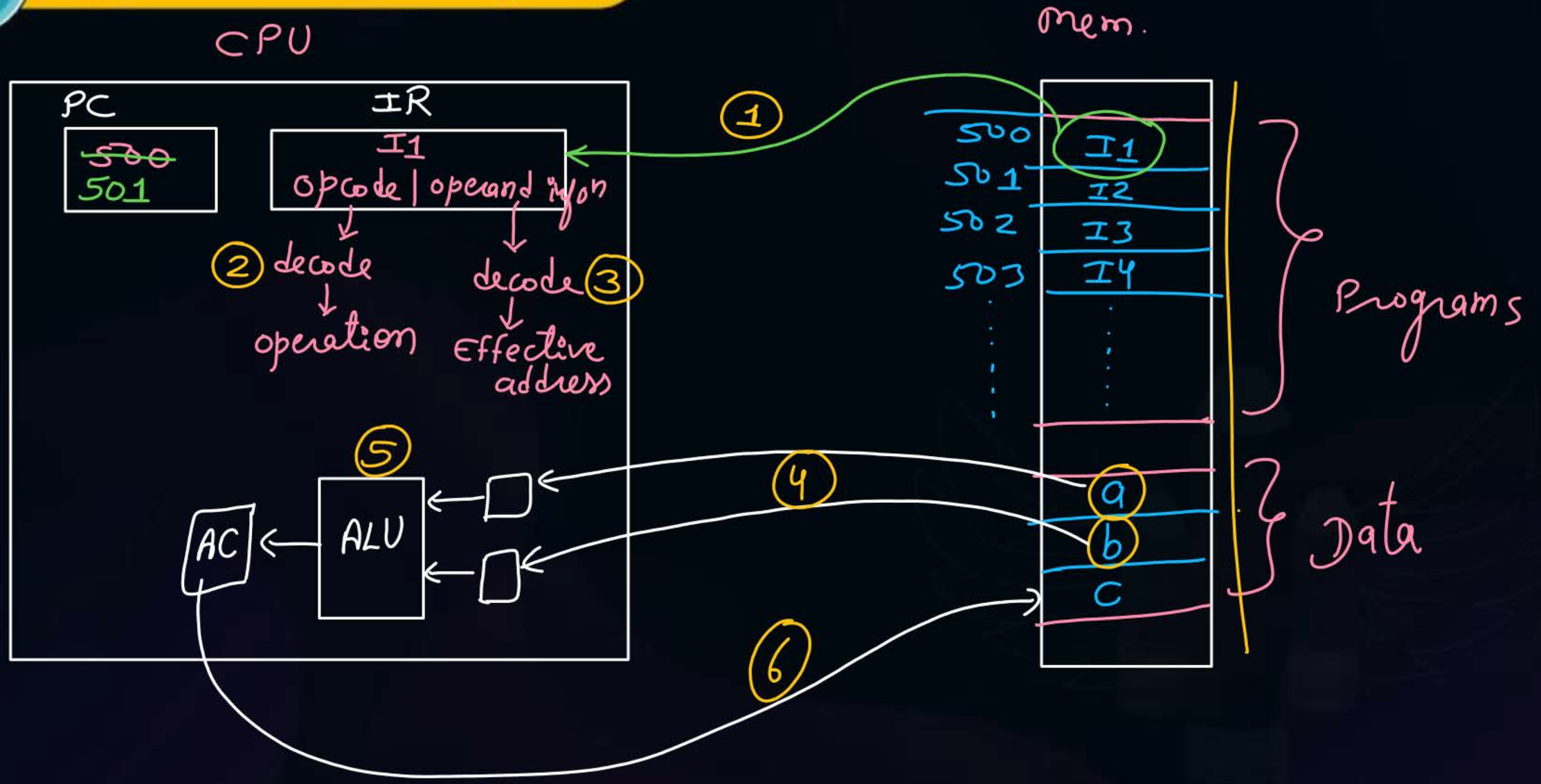
⇓

No any change in PC





Topic : Instruction Cycle





Topic : Instruction Cycle

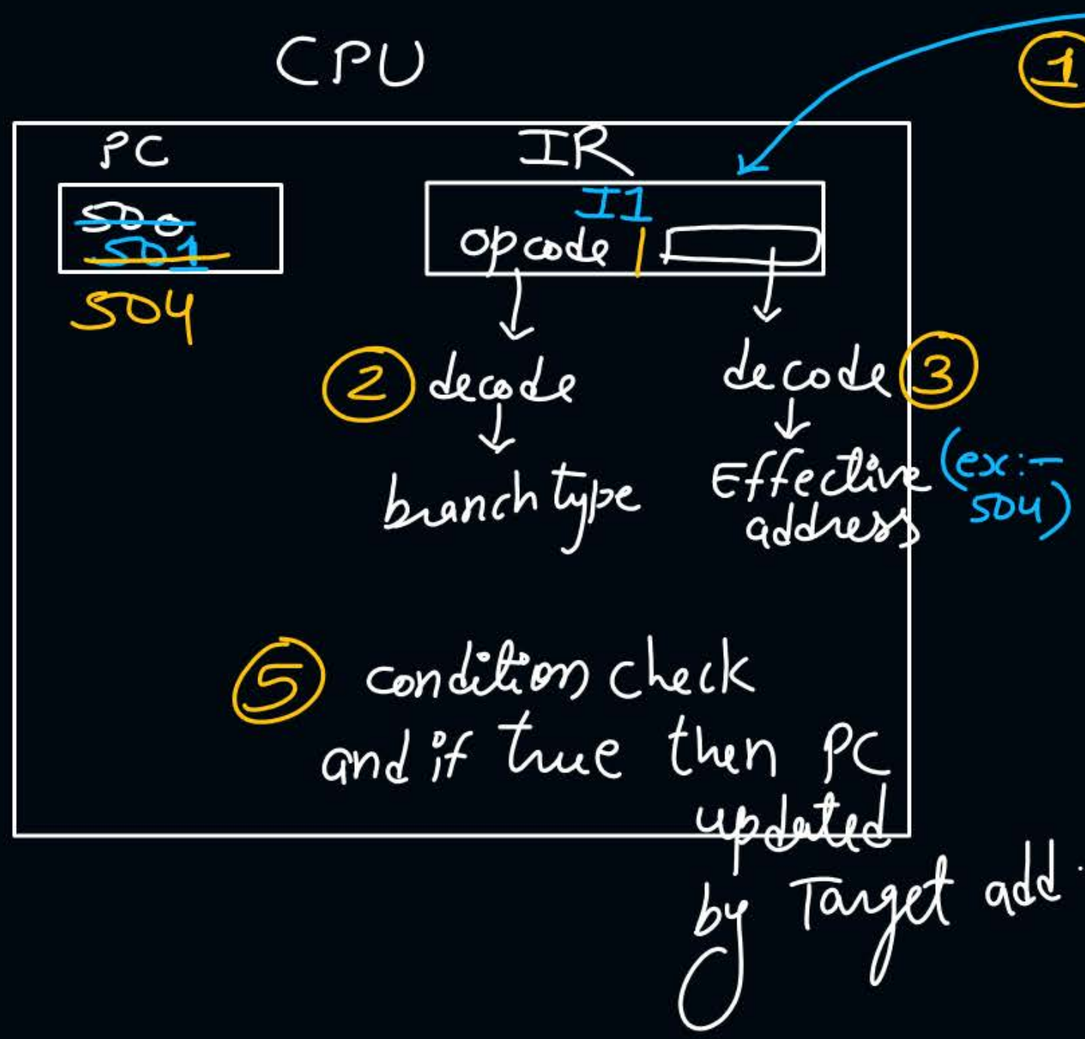
1. Instruction Fetch
2. Instruction Decode
3. Effective Address Calculation
4. Operand Fetch
5. Execution
6. Write Back Result

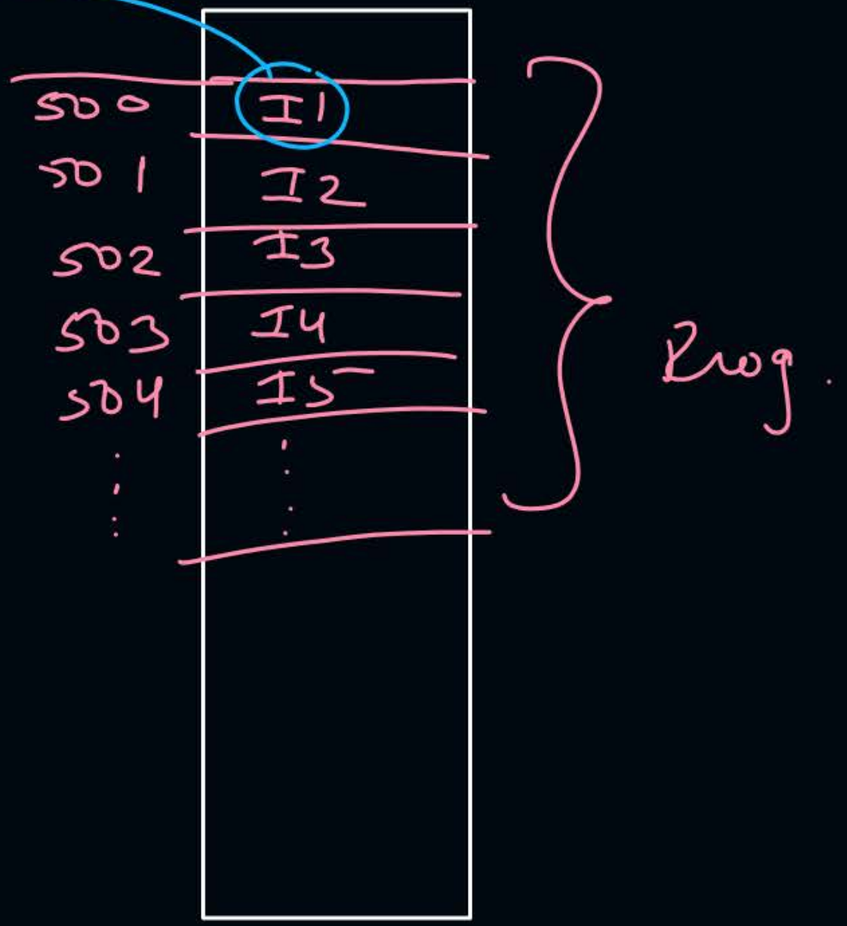


Topic : Fetch Cycle & Execution Cycle

Fetch Cycle:	Instruction Fetch
Execution Cycle:	Instruction Decode
	Effective Address Calculation
	Operand Fetch
	Execution
	Write Back the result s

Instⁿ cycle for Branch instⁿ :-



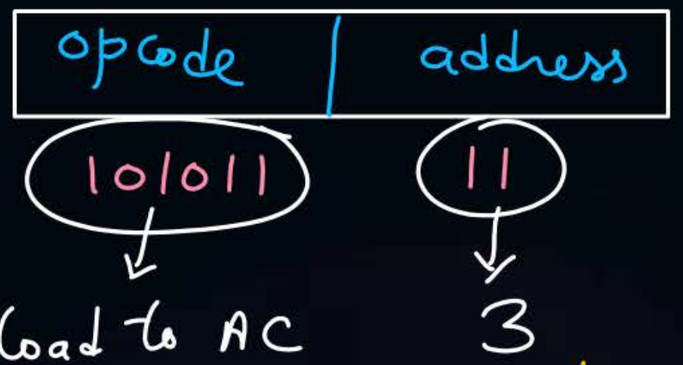




Topic : Computation vs Branch Type Instruction

	Computation Type	Branch Type
Instruction Fetch	→ fetch instⁿ from mem to IR → PC increment	} —— || ——
Instruction Decode	→ decodes opcode	→ decode opcode
Effective Address Calculation	→ calculates add. of operands	→ calculates target add.
Operand Fetch	→ fetch operands from source	X
Execution	→ Performs operatⁿ & generates result in AC	→ condition check and PC updated in condition is true
Write Back Result	→ writes back result to destinatⁿ from AC	X

Topic : Why Addressing Modes



Interpretation

↓

AC ← #3

or

AC ← R3

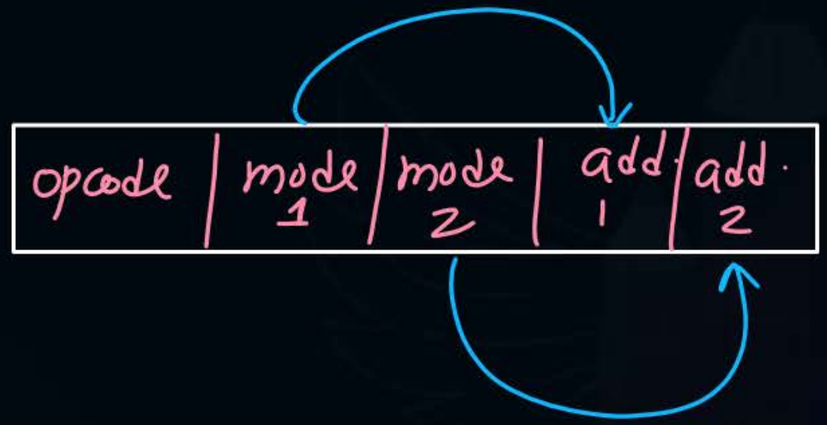
or

AC ← M[3]

Soln ⟹



2 – add. instn





Topic : Addressing Modes

- It specifies how and from where the operands are obtained for an instruction



2 mins Summary

Topic Effective Address

Topic Branch Instruction

Topic Instruction Cycle

Topic Fetch Cycle & Execution Cycle

Topic Addressing Modes



Happy Learning

THANK - YOU